Team Math Guide

Cheat sheet + full derivations for DCA-Trie

Bernard

July 27, 2026

Cheat Sheet — All Key Equations at a Glance

Notation

Symbol	Meaning	Plain English
$\mathcal{G}$	Knowledge graph	A network of facts (entities + relations)
$e \in \mathcal{E}$	Entity node	A named thing (e.g. "Paris", "France")
$r \in \mathcal{R}$	Relation	A typed edge (e.g. "capital_of")
$\vec{c}$	Reasoning chain	A path through the KG: $c_1 \xrightarrow{r_1} c_2 \xrightarrow{r_2} \cdots$
L	Chain length	Number of hops in the path
$\mathcal{N}(s)$	Neighbours of s	All nodes directly connected to s
$\mathcal{N}(s, r)$	Neighbours via r	Nodes connected to s by relation r
L_{lm}	LLM	The language model we use for generation
$\text{head}(i)$	Head entity at step i	The entity we are currently expanding
τ	Cosine threshold	Minimum similarity to accept an entity
$\mathcal{T}$	TypeOracle	KG-based type checker
rk_e	KG role of entity e	What types e plays (e.g. person, city)
tr_r	Range of relation r	What type the target must be
$T(\vec{c})$	Correctness gate	1 if all triples are in the KG
$C_{\max}$	Max valid chains	Total number of real KG paths of length L
B	Beam size	How many paths we keep in parallel
d_{avg}	Average out-degree	Mean number of neighbours per node
S	Score	How good a reasoning path is
S^*	Best score	Highest scoring valid answer

Key Equations

1. **Path probability (chain rule):**

$$P(\vec{c}) = \prod_{i=1}^{L} P(c_i \mid c_1, c_2, \ldots, c_{i-1})$$

2. **Graph-constrained logits:**

$$p_i(v) = \begin{cases} \text{softmax}(z_i)_v & \text{if } v \in \mathcal{N}(\text{head}(i)) \\ 0 & \text{otherwise} \end{cases}$$

3. **Cosine similarity (v1 scoring):**

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

4. **Decomposed product (v2 scoring):**

$$S(\vec{c}) = \prod_{i=1}^{L} P(c_i \mid \vec{c}_{<i}) \cdot T(\vec{c})$$

where $T(\vec{c}) = \prod_{i=1}^{L} \mathbb{1}[(\text{head}(i),\, r_i,\, c_{i+1}) \in \mathcal{G}]$.

5. **TypeOracle range gate:**

$$\text{range\_gate}(r, e) = \mathbb{1}\left[\text{tr}_r \subseteq \text{rk}_e\right]$$

6. **TypeOracle type gate:**

$$\text{type\_gate}(s, r) = \bigcup_{e \in \mathcal{N}(s,r)} \text{rk}_e$$

7. **Hits@k:**

$$\text{Hits@}k = \frac{|\{q : \text{rank}(q) \leq k\}|}{|\mathcal{Q}|}$$

8. **V1 complexity:**

$$O(E^L \cdot L)$$

where $E = \max_{s \in \mathcal{G}} |\mathcal{N}(s)|$.

9. **V2 complexity:**

$$O(L \cdot B \cdot d_{\text{avg}})$$

One-Liners for Defense

If they ask...	You say...
"What is graph-constrained decoding?"	"We mask the LLM's output at each step so it can only generate tokens that are valid edges in the knowledge graph."
"Why cosine similarity?"	"It measures how close the LLM's prediction is to the real KG embedding, without caring about magnitude—just direction."
"What does the TypeOracle do?"	"It's a semantic gate: before we let the LLM generate a relation, we check whether its target type is consistent with the entity's role in the KG."
"Why is v2 better than v1?"	"V1 scales exponentially with chain length $O(E^L)$. V2 scales linearly $O(L \cdot B \cdot d)$ by using beam search instead of enumerating all paths."
"What does $T(\vec{c})$ mean?"	"It's a correctness check: 1 if every triple in the chain actually exists in the KG, 0 otherwise. This is what makes DCA-Trie faithful."
"Is the output faithful?"	"Yes. We prove that the constrained decoder only outputs chains where every triple is in the KG. The proof uses mathematical induction on the chain length."

Full Derivations

0.1 1. Path Scoring — Why a Product?

Why do we need this? We need to assign a score to each reasoning path $\vec{c} = (c_1, r_1, c_2, r_2, \ldots, c_L)$. The score tells us how likely this path is according to the LLM. We want high-probability paths (the LLM is confident) to rank higher.

Step-by-step

Start with the joint probability.

We want $P(c_1, r_1, c_2, r_2, \ldots, c_L)$—the probability of the entire sequence of tokens.

Apply the chain rule of probability.

The chain rule says: the joint probability of a sequence equals the product of each element conditional on everything before it:

$$P(x_1, x_2, \ldots, x_n) = P(x_1) \cdot P(x_2 \mid x_1) \cdot P(x_3 \mid x_1, x_2) \cdots P(x_n \mid x_1, \ldots, x_{n-1})$$

This is always exact—no approximation. It's just probability theory.

Apply to our tokens.

Each step we generate either an entity name or a relation type. So:

$$P(\vec{c}) = \prod_{i=1}^{L} P(\text{entity}_i \mid \text{all previous tokens}) \cdot P(\text{relation}_i \mid \text{all previous tokens})$$

That's it.

The path score is just the LLM's probability of generating that exact sequence of tokens, step by step.

The code: The path score is computed in `dca_v2_generate()` at `experiments/type_oracle_full/decoding`. It multiplies the step-by-step LLM probabilities for each candidate path.

If asked about this, say: "The path score is just the LLM's own probability of generating that sequence—we multiply the step-by-step probabilities using the chain rule."

0.2 2. Graph-Constrained Logits — How We Enforce the KG

Why do we need this? Without constraints, the LLM can generate any token in its vocabulary (50,000+ tokens). We want to restrict it to only tokens that correspond to valid KG edges from the current entity.

Step-by-step

The LLM outputs logits.

At each step, the LLM produces a vector $z \in \mathbb{R}^{|V|}$ where $|V|$ is the vocabulary size. Each entry z_v is a raw score for token v.

We look up valid tokens.

Given the current head entity head(i), we find all neighbours in the KG:

$$\mathcal{N}(\text{head}(i)) = \{v : (\text{head}(i), r, v) \in \mathcal{G} \text{ for some } r\}$$

These are the only entities the LLM is allowed to mention next.

We mask invalid tokens.

We set the logits of invalid tokens to $-\infty$. After softmax, these become probability 0:

$$p_i(v) = \begin{cases} \text{softmax}(z_i)_v & \text{if } v \in \mathcal{N}(\text{head}(i)) \\ 0 & \text{otherwise} \end{cases}$$

Result: the LLM can only generate valid KG entities.

It's like a dropdown menu instead of a text box—you can only pick from what's actually in the graph.

The code: The mask is built in `GraphConstrainedDecoding` at `src/graph_constrained_decoding.py`. It calls `allowed_tokens_fn(head_entity, allowed_tokens_set)` which returns the IDs of valid next tokens.

If asked about this, say: "We set the logits of all invalid tokens to negative infinity so softmax makes their probability zero—the LLM physically cannot generate them."

0.3 3. Cosine Similarity — The v1 Gate

Why do we need this? The graph constraint ensures structural validity (the triple exists). But we also want semantic validity: does the LLM *mean* the same entity as the KG? Cosine similarity measures this.

Step-by-step

What is cosine similarity?

Given two vectors a and b:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2} \cdot \sqrt{\sum_i b_i^2}}$$

This measures the angle between the vectors, not their length. Range: $[-1, 1]$.

Why cosine, not Euclidean distance?

Because we care about direction, not magnitude. The LLM's internal representation of "Paris" might have different magnitude than the KG embedding, but if they point in the same direction, they represent the same concept.

How do we use it?

At each step, we:

1. Get the LLM's internal state for the entity it just generated.
2. Get the KG embedding of the candidate entity.
3. Compute cosine similarity.

4. If sim $> \tau$, we accept it; otherwise we reject it.

What is τ?

The threshold. We set it using the validation set. Typical range: 0.4–0.7. Too low = accept wrong entities. Too high = reject correct ones.

If asked about this, say: "Cosine similarity measures the angle between the LLM's prediction and the KG embedding—same direction means same meaning, regardless of magnitude."

0.4 4. Decomposed Product — The v2 Scoring Formula

Why do we need this? In v1, we scored entire paths at once. In v2, we score each step independently and multiply. This makes beam search possible and scales linearly instead of exponentially.

Step-by-step

Start with the path probability.

From Section 1:

$$P(\vec{c}) = \prod_{i=1}^{L} P(c_i \mid c_{<i})$$

Add the correctness gate.

We multiply by $T(\vec{c})$ which is 1 if every triple is in the KG, 0 otherwise:

$$S(\vec{c}) = P(\vec{c}) \cdot T(\vec{c}) = \left(\prod_{i=1}^{L} P(c_i \mid c_{<i}) \right) \cdot \left(\prod_{i=1}^{L} \mathbb{1}[(\text{head}(i), r_i, c_{i+1}) \in \mathcal{G}] \right)$$

Why the product of indicators?

Each indicator checks one triple. If any triple is missing from the KG, the whole product is 0. This is how we guarantee faithfulness—a single invalid triple kills the score.

Why decompose instead of scoring the whole path?

Because we can now do beam search: at each step, keep only the top-B partial paths. We don't need to enumerate all E^L paths.

The conditional independence assumption.

We assume each step depends only on the entities so far, not on the full token sequence. This is approximate but practical—the LLM's hidden state captures the history.

If asked about this, say: "The score is the product of step-by-step probabilities, multiplied by a correctness gate that's 0 if any triple is fake—this makes the decoder faithful by construction."

0.5 5. TypeOracle Gates — Semantic Pruning

Why do we need this? The graph constraint ensures the triple exists. The TypeOracle goes further: it checks whether the target entity's *type* makes sense for the relation. This prunes bad candidates before the LLM even sees them, reducing the search space.

Step-by-step

Setup: types in the KG.

Every entity e has a set of types rk_e (e.g. Paris: {City, Place}). Every relation r has a range type tr_r (e.g. capital_of: {Country}).

The range gate.

Given a relation r and a candidate entity e, accept only if e's type is compatible with r's range:

$$\text{range\_gate}(r, e) = \mathbb{1}[\text{tr}_r \subseteq \text{rk}_e]$$

Example: capital_of requires Country type. "Paris" has type City—gate rejects it. "France" has type Country—gate accepts it.

The type gate.

Given a head entity s and relation r, compute what types the target could be:

$$\text{type\_gate}(s, r) = \bigcup_{e \in \mathcal{N}(s,r)} \text{rk}_e$$

This is the union of types of all actual neighbours of s via r. It tells the LLM: "the next entity must be one of these types."

How they work together.

At each step:

1. Compute type_gate(head, r) = what types are allowed.
2. For each candidate entity e, check range_gate(r, e) = is e's type in the allowed set?
3. Only entities passing both gates are offered to the LLM.

Worked example.

Head = "Paris", relation = "capital_of":

- type_gate: neighbours of Paris via capital_of = {France}. Types = {Country}.
- Candidate "France": range_gate checks Country ∈ {Country} → **accept**.
- Candidate "Berlin": range_gate checks Country ∈ {City} → **reject**.

If asked about this, say: "The TypeOracle is a semantic filter: it checks whether the candidate entity's type makes sense for the relation, pruning nonsensical paths before the LLM even considers them."

0.6 6. Complexity Analysis — Why v2 Scales

Why do we need this? This is the core argument for why our approach works on real KGs. V1 hits a wall at chain length 3–4. V2 scales linearly.

Step-by-step

V1: enumerate all paths.

For each of L steps, try all E neighbours. Total paths: E^L. For each path, score it in $O(L)$ time. So:

$$\text{V1} = O(E^L \cdot L)$$

Example: $E = 100$, $L = 3$. That's $100^3 = 1{,}000{,}000$ paths. Feasible. But $L = 4$? 100,000,000. Not feasible.

V2: beam search.

At each of L steps, expand B beams. Each beam expands to d_{avg} candidates. Score each in $O(1)$ (we track the running product). So:

$$\text{V2} = O(L \cdot B \cdot d_{\text{avg}})$$

Example: $L = 4$, $B = 5$, $d_{\text{avg}} = 10$. That's $4 \times 5 \times 10 = 200$ operations. Always feasible.

Where the savings come from.

V1 is exponential because each step multiplies the number of paths. V2 is linear because we keep only the top-B at each step—we never let the number of paths grow beyond B.

What if B is too small?

We might miss the correct answer. This is the precision-recall tradeoff of beam search. Typical: $B = 3$–10.

If asked about this, say: "V1 tries all paths—exponential. V2 keeps only the best B at each step—linear. That's why it scales to real knowledge graphs."

0.7 7. Beam Search — How We Find the Best Path

Why do we need this? Beam search is how we go from "enumerate everything" (v1) to "keep the best candidates" (v2). It's the algorithm that makes v2 practical.

Step-by-step

The idea.

At each step:

1. Take each beam (partial path).
2. Expand it to all valid next entities (using the constrained LLM).
3. Score each expansion.
4. Keep only the top B across all beams.

The dynamic trie (DoG contribution).

Instead of pre-computing all valid paths, we build a trie on-the-fly as the LLM generates. Each beam has its own trie that grows as new triples are generated.

Head pool (DoG contribution).

Each beam tracks which entities have been mentioned so far (head(i)). This prevents cycles: you can't go back to an entity you already visited.

Stopping condition.

We stop when:

- The LLM generates the L-th hop, or
- No valid expansions exist (dead end), or
- The LLM generates an end-of-sequence token.

What we return.

The beam with the highest score $S(\vec{c})$.

If asked about this, say: "Beam search keeps the best B partial paths at each step, expanding only the most promising ones—it's like keeping a shortlist that gets refined at each hop."

0.8 8. Hits@k — How We Measure Success

Why do we need this? We need a single number to say "how good is our method?" Hits@k tells us: out of all test questions, in what fraction was the correct answer in our top-k predictions?

Step-by-step

Definition.

For a test question q, let $\text{rank}(q)$ be the position of the correct answer when we sort all candidate answers by score. Then:

$$\text{Hits@}k = \frac{|\{q \in \mathcal{Q} : \text{rank}(q) \leq k\}|}{|\mathcal{Q}|}$$

In words: "what fraction of questions have the correct answer in the top k?"

Hits@1.

Is the *very best* prediction correct? This is the strictest metric.

Hits@3 / Hits@10.

Is the correct answer *somewhere* in the top 3 or top 10? More forgiving.

What numbers are good?

On WebQSP:

- Baseline (no KG): ∼0.3
- GCR (ICML 2025): ∼0.45–0.55
- DCA-Trie v1: ∼0.50–0.60
- DCA-Trie v2: ∼0.55–0.65 (expected)

Why Hits@1, not accuracy?

Because we might generate multiple candidate answers. Hits@1 checks if the top one is right. Accuracy would require a single binary judgment per question.

If asked about this, say: "Hits@1 is the fraction of test questions where our top prediction is correct—it's the headline number for how well the method works."

0.9 9. Faithfulness Theorem — Why the Output is Always Correct

Why do we need this? This is what makes DCA-Trie different from just asking an LLM a question. We can *prove* the output is grounded in the KG.

Step-by-step

What we claim.

The constrained decoder only outputs chains $\vec{c}$ where every triple is in the KG:

$$T(\vec{c}) = \prod_{i=1}^{L} \mathbb{1}[(\text{head}(i), r_i, c_{i+1}) \in \mathcal{G}] = 1$$

Proof by induction on L.

Base case ($L = 1$): We only offer tokens from $\mathcal{N}(\text{head}(0))$. So $(c_1, r_1, c_2) \in \mathcal{G}$ by construction.

Inductive step: Assume the decoder is faithful for chains of length k. For length $k + 1$:

1. By the inductive hypothesis, $\text{head}(k) = c_k$ and all previous triples are in $\mathcal{G}$.
2. At step $k + 1$, the decoder only offers tokens from $\mathcal{N}(\text{head}(k))$.
3. So $(c_k, r_k, c_{k+1}) \in \mathcal{G}$.
4. Therefore $T(\vec{c}) = 1$ for all steps.

What this means in practice.

If we use the constrained decoder, we can never generate a hallucinated triple. Every fact in the output chain is verified against the KG.

If asked about this, say: "We prove by induction that the decoder can only generate chains where every triple is in the knowledge graph—hallucination is structurally impossible."